

第七章：中间代码生成(2)

表达式的中间代码生成
下标变量的中间代码生成
赋值语句的中间代码生成

1. 表达式的中间代码生成

- ◆ 表达式的中间代码就是正确计算表达式值的四元式序列。
- ◆ 表达式中的变量可以是简单、复杂变量（数组变量，结构体成员变量等），还可以是函数调用，而表达式中的运算符也可以包括算术运算符，布尔运算符等。
- ◆ 本小节将给出简单算术表达式的中间代码生成的LL(1)语法制导方法。

基于LL(1)方法

$$\text{【1】 } E \rightarrow T$$

$$\text{【2】 } E \rightarrow E + T$$

$$\text{【3】 } E \rightarrow E - T$$

$$\text{【4】 } T \rightarrow P$$

$$\text{【5】 } T \rightarrow T * P$$

$$\text{【6】 } T \rightarrow T / P$$

$$\text{【7】 } P \rightarrow (E)$$

$$\text{【8】 } P \rightarrow id$$

$$\text{【9】 } P \rightarrow C$$

$$(1) E \rightarrow T E_s$$

$$(2) E_s \rightarrow \varepsilon$$

$$(3) E_s \rightarrow + T E_s$$

$$(4) E_s \rightarrow - T E_s$$

$$(5) T \rightarrow P T_s$$

$$(6) T_s \rightarrow \varepsilon$$

$$(7) T_s \rightarrow * P T_s$$

$$(8) T_s \rightarrow / P T_s$$

$$(9) P \rightarrow C$$

$$(10) P \rightarrow id$$

$$(11) P \rightarrow (E)$$

原文法存在左递归，不符合LL(1)文法条件，消除左递归后文法变换为右侧文法，经计算Predict集，满足LL(1)文法条件。

简单表达式的LL(1)分析表

	C	id	(	)	+	-	*	/	#
E	1	1	1						
Es				2	3	4			2
T	5	5	5						
Ts				6	6	6	7	8	6
P	9	10	11						

产生式	Predict集
(1) $E \rightarrow T E_s$	C, id, (
(2) $E_s \rightarrow \varepsilon$	#,)
(3) $E_s \rightarrow + T E_s$	+
(4) $E_s \rightarrow - T E_s$	-
(5) $T \rightarrow P T_s$	C, id, (
(6) $T_s \rightarrow \varepsilon$	#,), +, -
(7) $T_s \rightarrow * P T_s$	*
(8) $T_s \rightarrow / P T_s$	/
(9) $P \rightarrow C$	C
(10) $P \rightarrow id$	Id
(11) $P \rightarrow (E)$	(

•简单表达式的带有动作符的LL(1)文法

- (1) $E \rightarrow T E_s$
- (2) $E_s \rightarrow \varepsilon$
- (3) $E_s \rightarrow +T \# \text{GenCode}(+) \# E_s$
- (4) $E_s \rightarrow -T \# \text{GenCode}(-) \# E_s$
- (5) $T \rightarrow P T_s$
- (6) $T_s \rightarrow \varepsilon$
- (7) $T_s \rightarrow *P \# \text{GenCode}(*) \# T_s$
- (8) $T_s \rightarrow /P \# \text{GenCode}(/) \# T_s$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow id \# \text{Push}(id) \#$
- (11) $P \rightarrow (E)$

- ◆ 当遇到常量C和简单变量id时，把它们的语义信息压入语义栈；
- ◆ 当处理完一个运算符(+, -, *, /)的右分量时，该运算符的左、右运算分量已经分别存放在语义栈Sem的次栈顶和栈顶的位置，因此可以生成相应的运算符的四元式，并把运算结果的语义信息压入语义栈。

表达式 $x + y * z$ 的中间代码生成过程(1)

分析栈S

输入流T

动作

# E	x + y * z #	LL[E ,id] = [1]
# EsT	x + y * z #	LL [T ,id] = [5]
# EsTs P	x + y * z #	LL [P ,id] = [10]
# EsTs #Push (id)# id	x + y * z #	Match
# EsTs #Push (id)#	+ y * z #	#Push (id)#
# EsTs	+ y * z #	

Top Top	intptr (x.level, x.off, dir)

语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode} (+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode} (-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode} (*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode} (/) \# Ts$
- (9) $P \rightarrow C \# \text{Push} (C) \#$
- (10) $P \rightarrow id \# \text{Push} (id) \#$
- (11) $P \rightarrow (E)$

表达式 $x + y * z$ 的中间代码生成过程(2)

分析栈S

输入流T

动作

EsTs

+ y * z #

LL [Ts,+] = [6]

#Es

+ y * z #

LL [Es,+] = [3]

Es #GenCode (+)# T +

+ y * z #

Match

Es #GenCode (+)# T

y * z #

LL [T,id] = [5]

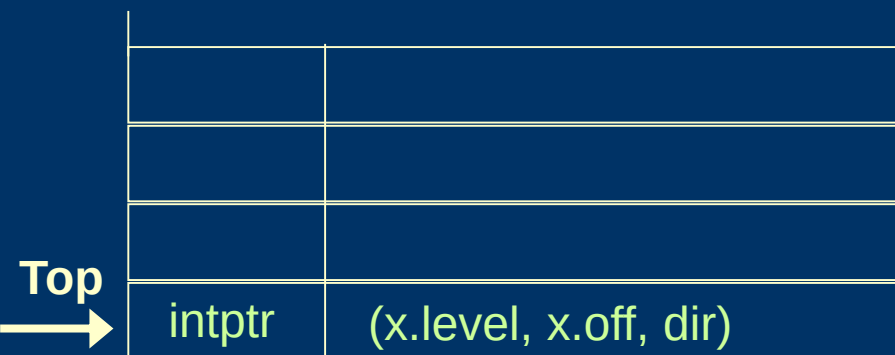
Es #GenCode (+)#TsP

y * z #

LL [P,id] = [10]

Es #GenCode (+)# Ts #Push (id)# id

y * z #



语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode} (+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode} (-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode} (*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode} (/) \# Ts$
- (9) $P \rightarrow C \# \text{Push} (C) \#$
- (10) $P \rightarrow id \# \text{Push} (id) \#$
- (11) $P \rightarrow (E)$

表达式 $x + y * z$ 的中间代码生成过程(3)

分析栈S

输入流T

动作

Es# **GenCode (+)** # Ts # **Push (id)** # id

y * z #

Match

Es# **GenCode (+)** # Ts # **Push (id)**

* z #

Push (id)

Es # **GenCode (+)** # Ts

* z #

Top Top	intptr	(y.level, y.off, dir)
	intptr	(x.level, x.off, dir)

语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode}(+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode}(-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode}(*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode}(/) \# Ts$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow id \# \text{Push}(id) \#$
- (11) $P \rightarrow (E)$

表达式 $x + y * z$ 的中间代码生成过程(4)

分析栈S

输入流T

动作

Es # **GenCode (+)** # Ts

* z #

LL [Ts,*] = [7]

Es # **GenCode (+)** # Ts # **GenCode (*)** # P*

* z #

Match

Es # **GenCode (+)** # Ts # **GenCode (*)** # P

z #

Top →		
	intptr	(y.level, y.off, dir)
	intptr	(x.level, x.off, dir)

语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode } (+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode } (-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode } (*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode } (/) \# Ts$
- (9) $P \rightarrow C \# \text{Push } (C) \#$
- (10) $P \rightarrow id \# \text{Push } (id) \#$
- (11) $P \rightarrow (E)$

表达式 $x + y * z$ 的中间代码生成过程(5)

分析栈S

输入流T

动作

Es # **GenCode (+)** # Ts # **GenCode (*)** # P

z #

LL[P,id] = [10]

Es # **GenCode (+)** # Ts # **GenCode (*)** # **Push (id)** # id

z #

Match

Es # **GenCode (+)** # Ts # **GenCode (*)** # **Push (id)**

#

Top →		
	intptr	(y.level, y.off, dir)
	intptr	(x.level, x.off, dir)

语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode} (+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode} (-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode} (*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode} (/) \# Ts$
- (9) $P \rightarrow C \# \text{Push} (C) \#$
- (10) $P \rightarrow id \# \text{Push} (id) \#$
- (11) $P \rightarrow (E)$

表达式 $x + y * z$ 的中间代码生成过程(6)

分析栈S

输入流T

动作

# Es # GenCode (+) # Ts # GenCode (*) # # Push (id) #	#	#Push (id)#
# Es # GenCode (+) # Ts # GenCode (*) #	#	#GenCode (*)#
# Es # GenCode (+) # Ts	#	

(MULTI, (y.level, y.off, dir), (z.level, z.off, dir), (-1, t_1 , dir))

intptr	(z.level, z.off, dir)	← Top
intptr	(y.level, y.off, dir)	← Top
intptr	(x.level, x.off, dir)	

语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode } (+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode } (-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode } (*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode } (/) \# Ts$
- (9) $P \rightarrow C \# \text{Push } (C) \#$
- (10) $P \rightarrow id \# \text{Push } (id) \#$
- (11) $P \rightarrow (E)$

表达式 $x + y * z$ 的中间代码生成过程(7)

分析栈S

输入流T

动作

Es # **GenCode (+)** # Ts
Es # **GenCode (+)** #
Es
#

LL [Ts, #] = [6]
#GenCode (+)#
LL [Es, #] = [2]
Success

(MULTI, y, z, t₁)

(ADDI, x, t₁, t₂)

(MULTI, (y.level, y.off, dir), (z.level, z.off, dir), (-1, t₁, dir))

(ADDI, (x.level, x.off, dir), (-1, t₁, dir), (-1, t₂, dir))

intptr	(-1, t ₁ , dir)	Top ↑ Top ↑
intptr	(-1, t ₂ , dir)	

语义栈Sem

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \epsilon$
- (3) $Es \rightarrow + T \# \text{GenCode}(+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode}(-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \epsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode}(*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode}(/) \# Ts$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow id \# \text{Push}(id) \#$
- (11) $P \rightarrow (E)$

例：将下列表达式翻译成四元式序列。

$$X*2+A*(i+1)/(j+1)$$

其中 i 和 j 为整型变量，其它为实型变量。

1. (FLOAT, 2, _, t₁)
2. (MULTF, X, t₁, t₂)
3. (ADDI, i, 1, t₃)
4. (FLOAT, t₃, _, t₄)
5. (MULTF, A, t₄, t₅)
6. (ADDI, j, 1, t₆)
7. (FLOAT, t₆, _, t₇)
8. (DIVF, t₅, t₇, t₈)
9. (ADDF, t₂, t₈, t₉)

2. 下标变量的中间代码生成

2.1 下标变量的地址计算

假设有数组类型的变量声明如下：

$A : \text{array } [L_1..U_1] [L_2 .. U_2] \dots [L_n .. U_n] \text{ of } T ; (n \geq 1)$

◆ 数组A占单元大小： $\text{size}(A) = D_1 * D_2 * \dots * D_n * \alpha$

其中 $D_i = U_i - L_i + 1$, α 为类型T所占单元数。

◆ 下标变量取址： $a[i_1][i_2][i_3] \dots [i_n]$

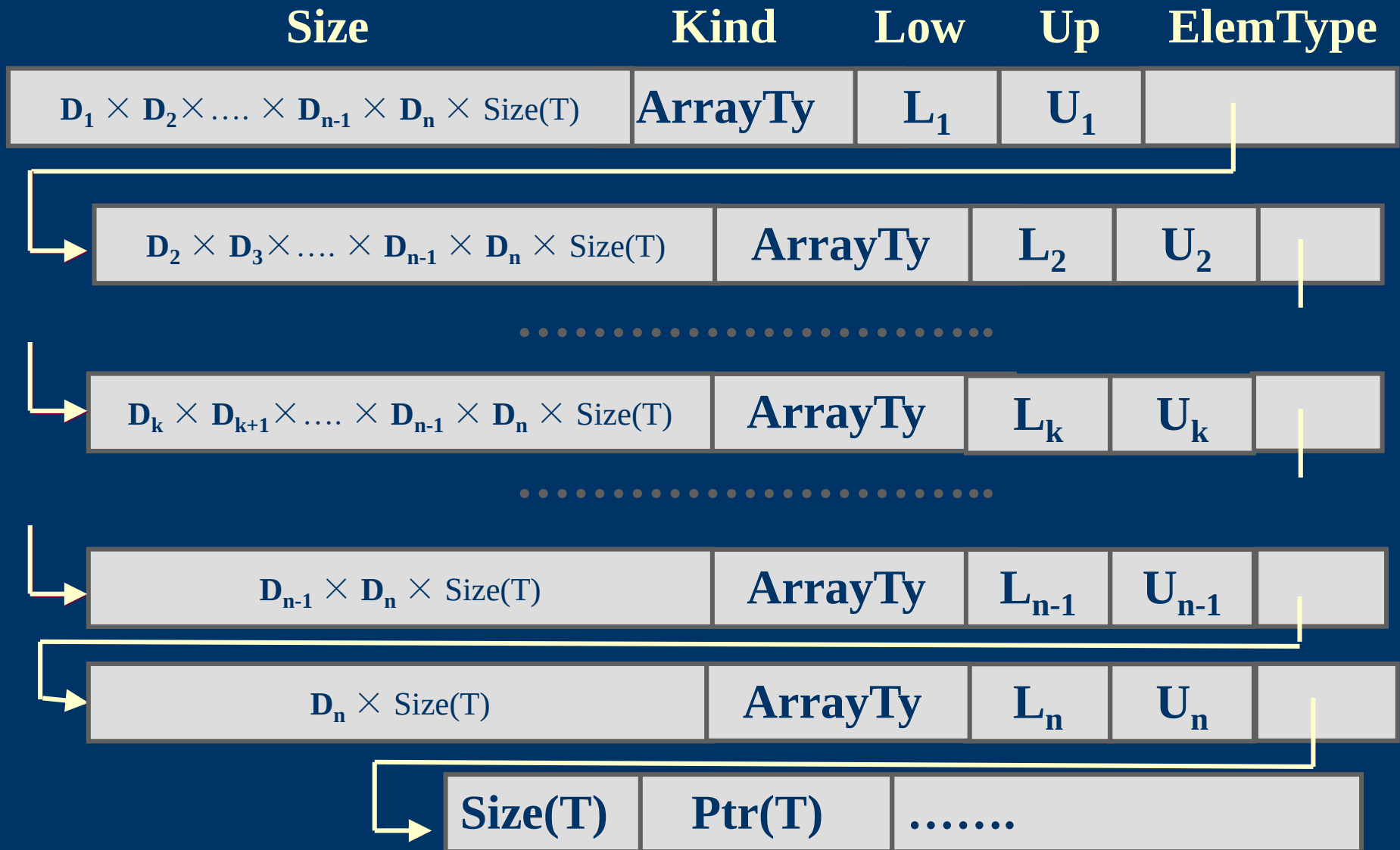
$= \text{addr}(a) + (i_1 - L_1) * S_1 + (i_2 - L_2) * S_2 + \dots + (i_n - L_n) * S_n$

//注：此处的 S_i 不同于书中P130给出的 S_i , 为第i层数组变量元素空间大小

假设有数组类型的变量声明如下：

$A : \text{array } [L_1..U_1] [L_2..U_2] \cdots [L_n..U_n] \text{ of } T ; (n \geq 1)$

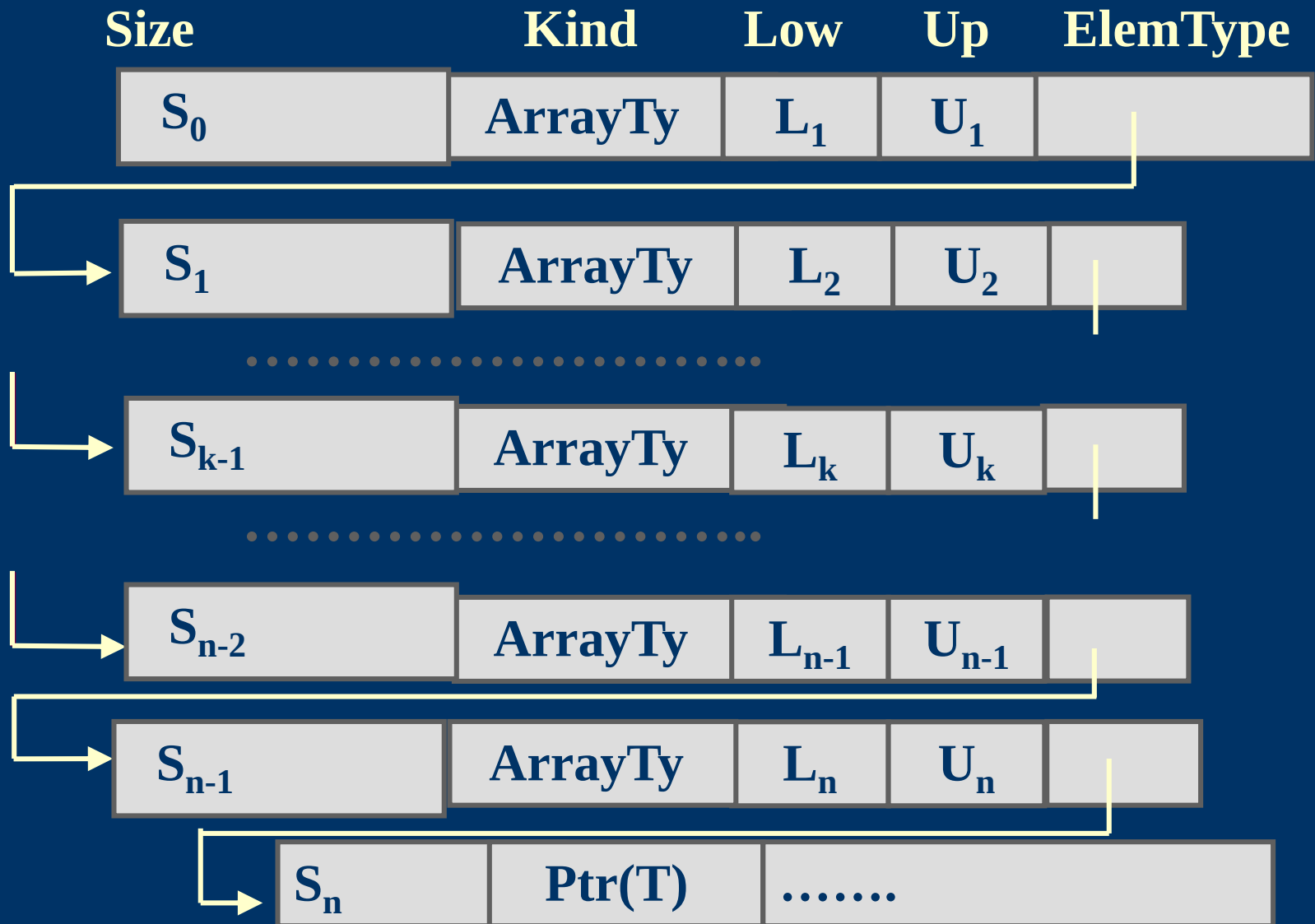
数组类型的内部表示 其中 $D_i = U_i - L_i + 1$



假设有数组类型的变量声明如下：

$A : \text{array } [L_1..U_1] [L_2 .. U_2] \cdots [L_n .. U_n] \text{ of } T ; (n \geq 1)$

将size转化为S则有：



设起始地址为 off_A

`int A[D1][D2];`

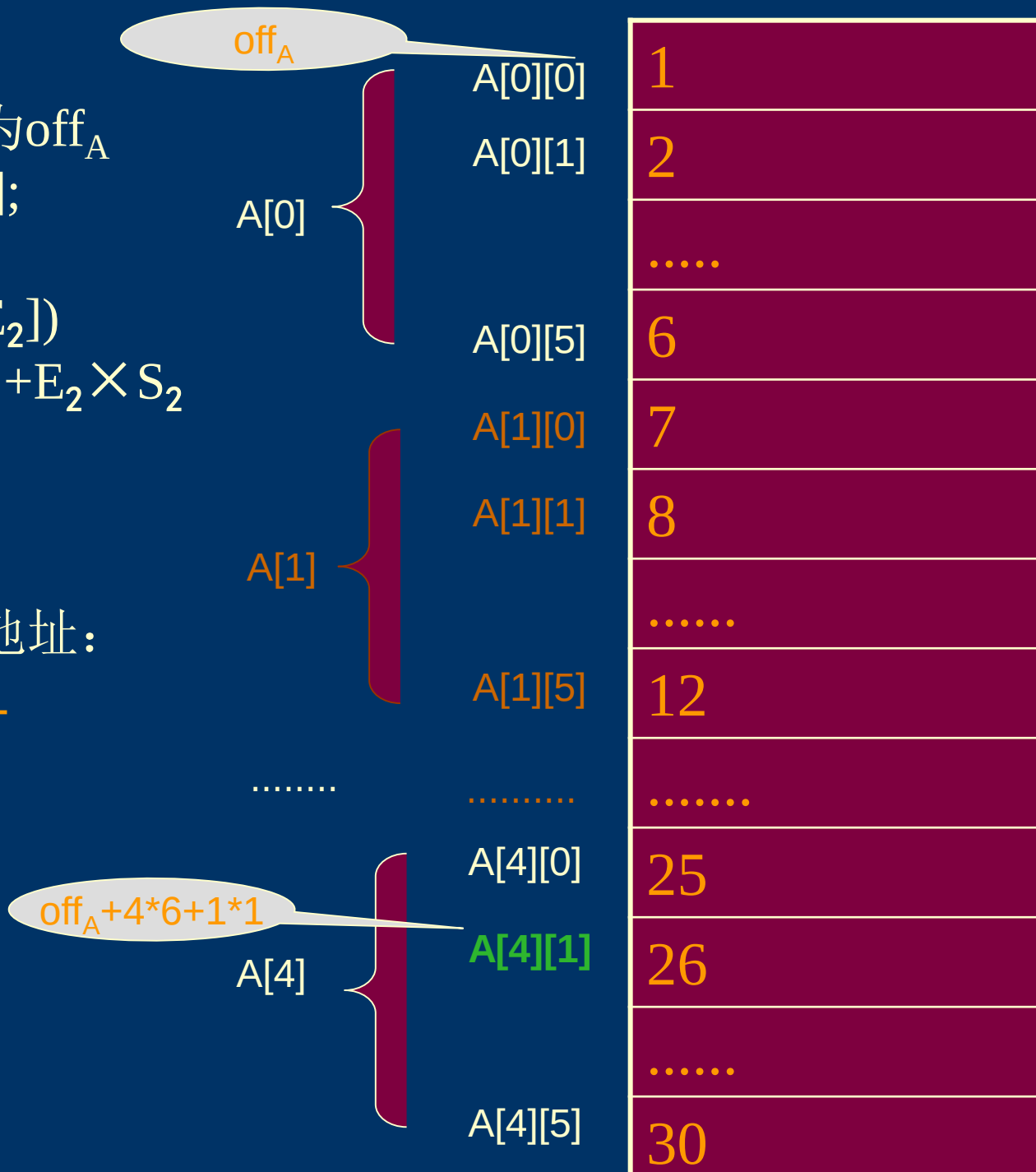
$\text{Addr}(A[E_1][E_2])$

$$= \text{off}_A + E_1 \times S_1 + E_2 \times S_2$$

$$S_1 = D_2, S_2 = 1$$

则 $A[4][1]$ 的地址:

$$\text{off}_A + 4 \times 6 + 1 \times 1$$



2. 2下标变量的四元式结构

- ◆ 对于任意下标变量 $A[E_1][E_2] \dots [E_k]$
($n \geq k$)，下面给出一种直接的中间代码结构：

$E_1 \rightarrow t_1$

- ◆ (SUBI, t_1 , L_1 , t_2)
- ◆ (MULTI, t_2 , S_1 , t_3)
- ◆ (AADD, A , t_3 , t_4)

$A[E_1]$ 地址

◆ $E_2 \rightarrow t_5$

◆ (SUBI, t_5 , L_2 , t_6)

◆ (MULTI, t_6 , S_2 , t_7)

◆ (AADD, t_4 , t_7 , t_8)

◆

$A[E_1] [E_2]$ 地址

◆ $E_k \rightarrow t_n$

◆ (SUBI, t_n , L_k , t_{n+1})

◆ (MULTI, t_{n+1} , S_k , t_{n+2})

◆ (AADD, $t_?$, t_{n+2} , t_{n+3})

$A[E_1] [E_2] \dots [E_k]$ 地址

练习题

$i, j: \text{integer};$

$a: \text{array}[1..10][1..5] \text{ of integer};$

下标变量 $a[i+1][j*i-2]$ 的中间代码:

1. $(+, i, 1, t_1)$
2. $(-, t_1, 1, t_2)$
3. $(*, t_2, 5, t_3)$
4. $([], a, t_3, t_4)$
5. $(*, j, i, t_5)$
6. $(-, t_5, 2, t_6)$
7. $(-, t_6, 1, t_7)$
8. $(*, t_7, 1, t_8)$
9. $([], t_4, t_8, t_9)$

2.3 下标变量的中间代码生成

下标变量的LL(1)动作文法可以写成：

(1) $V \rightarrow \text{\#Init\#id\#PUSH(id)\#A}$

(2) $A \rightarrow [E] \text{\#AddNext\#B}$

(3) $B \rightarrow \varepsilon$

(4) $B \rightarrow [E] \text{\#AddNext\#B}$

其中:

- ◆ Init用来初始化下标计数器 $k = 0$;
- ◆ 遇到变量标识符, 则调用PUSH(id)把其语义信息 (一般为类型和地址) 入栈中;
- ◆ 在下标表达式之后, 调用AddNext, 用于生成地址累加的四元式。它的算法是:

AddNext算法:

- 下标计数器加1, $k := k+1$;
- 从语义栈中取出下标表达式计算结果result;
- 生成计算第k维下标需要累加的地址偏移的四元式组:
(-, result, L_k , t_{n+1})
(*, t_{n+1} , S_k , t_{n+2})
- 从语义栈中取出地址累加结果 $t_?$;
- 生成地址累加四元式($[]$, $t_?$, t_{n+2} , t_{n+3})
 - ① pop (2) ;
 - ② 把累加后的地址结果 t_{n+3} 压入语义栈;

扩展表达式的带有动作符的LL(1)文法

- (1) $E \rightarrow TEs$
- (2) $Es \rightarrow \varepsilon$
- (3) $Es \rightarrow +TEs$
- (4) $Es \rightarrow -TEs$
- (5) $T \rightarrow PTs$
- (6) $Ts \rightarrow \varepsilon$
- (7) $Ts \rightarrow *PTs$
- (8) $Ts \rightarrow /PTs$
- (9) $P \rightarrow C$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow idA$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow [E]B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E]B$

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \varepsilon$
- (3) $Es \rightarrow + T \# \text{GenCode} (+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode} (-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \varepsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode} (*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode} (/) \# Ts$
- (9) $P \rightarrow C \# \text{Push} (C) \#$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow id \# \text{Push} (id) \# A$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow \# \text{init} \# [E] \# \text{AddNext} \# B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E] \# \text{AddNext} \# B$

•简单算术表达式的LL(1)文法定义

- (1) $E \rightarrow TEs$
- (2) $Es \rightarrow \varepsilon$
- (3) $Es \rightarrow +TEs$
- (4) $Es \rightarrow -TEs$
- (5) $T \rightarrow PTs$
- (6) $Ts \rightarrow \varepsilon$
- (7) $Ts \rightarrow *PTs$
- (8) $Ts \rightarrow /PTs$
- (9) $P \rightarrow C$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow idA$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow [E]B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E]B$

	First集	Follow集
E	C, id , (	# ,) ,]
Es	ε , +, -	# ,),]
T	C, id , (	+, - , # ,),]
Ts	ε , *, /	+, -, # ,),]
P	C, id , (	*, / , +, -, # ,),]
V	id	*, / , +, -, # ,),]
A	ε , [	*, / , +, -, # ,),]
B	ε , [	*, / , +, -, # ,),]

•简单表达式的带有动作符的LL(1)分析表

	C	id	(	)	+	-	*	/	[	]	#
E	1	1	1								
Es				2	3	4				2	2
T	5	5	5								
Ts				6	6	6	7	8		6	6
P	9	10	11								
V		12									
A				13	13	13	13	13	14	13	13
B				15	15	15	15	15	16	15	15

产生式	Predict集
(1) $E \rightarrow T E_s$	C, id, (
(2) $E_s \rightarrow \varepsilon$	#,),]
(3) $E_s \rightarrow + T E_s$	+
(4) $E_s \rightarrow - T E_s$	-
(5) $T \rightarrow P T_s$	C, id, (
(6) $T_s \rightarrow \varepsilon$	#,), +, -,]
(7) $T_s \rightarrow * P T_s$	*
(8) $T_s \rightarrow / P T_s$	/
(9) $P \rightarrow C$	C
(10) $P \rightarrow V$	id
(11) $P \rightarrow (E)$	(
(12) $V \rightarrow idA$	id
(13) $A \rightarrow \varepsilon$	#,), +, -, *, /,]
(14) $A \rightarrow [E]B$	[
(15) $B \rightarrow \varepsilon$	#,), +, -, *, /,]
(16) $B \rightarrow [E]B$	[

分析栈S

输入流T

动作

#E	a[i][j]#	LL[E,id]=1
#E _s T	a[i][j]#	LL[T,id]=5
#E _s T _s P	a[i][j]#	LL[P,id]=10
#E _s T _s V	a[i][j]#	LL[V,id]=12
#E _s T _s A#PUSH(id)#id	a[i][j]#	MATCH
#E _s T _s A#PUSH(id)#	[i][j]#	PUSH(id)
#E _s T _s A	[i][j]#	LL[A,[]]=14

a

- (1) $E \rightarrow T E_s$
- (2) $E_s \rightarrow \varepsilon$
- (3) $E_s \rightarrow + T \# \text{GenCode}(+) \# E_s$
- (4) $E_s \rightarrow - T \# \text{GenCode}(-) \# E_s$
- (5) $T \rightarrow P T_s$
- (6) $T_s \rightarrow \varepsilon$
- (7) $T_s \rightarrow * P \# \text{GenCode}(*) \# T_s$
- (8) $T_s \rightarrow / P \# \text{GenCode}(/) \# T_s$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow id \# \text{Push}(id) \# A$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow \#init\# [E] \# \text{AddNext}\# B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E] \# \text{AddNext}\# B$

分析栈S

输入流T

动作

#E _S T _S A	[i][j]#	LL[A,[]]=14
#E _S T _S B #AddNext#] E [#init#	[i][j]#	init
#E _S T _S B #AddNext#] E[	[i][j]#	match
#E _S T _S B #AddNext#]E	i][j]#	LL[E,id]=1
#E _S T _S B #AddNext#]EsT	i][j]#	LL[T,id]=5
#E _S T _S B #AddNext#]EsT _S P	i][j]#	LL[P,id]=10
#E _S T _S B #AddNext#]EsT _S V	i][j]#	LL[V,id]=12
#E _S T _S B #AddNext#]EsT _S A#PUSH(id)#id	i][j]#	match

k=0



- (1) E → T Es
- (2) Es → ε
- (3) Es → + T # GenCode (+) # Es
- (4) Es → - T # GenCode (-) # Es
- (5) T → P Ts
- (6) Ts → ε
- (7) Ts → * P # GenCode (*) # Ts
- (8) Ts → / P # GenCode (/) # Ts
- (9) P → C # Push (C) #
- (10) P → V
- (11) P → (E)
- (12) V → id # Push (id) # A
- (13) A → ε
- (14) A → #init# [E] #AddNext# B
- (15) B → ε
- (16) B → [E] #AddNext# B

分析栈S

输入流T

动作

#E _S T _S B #AddNext#]EsT _S A#PUSH(id)#id	i[j]#	match
#E _S T _S B #AddNext#]EsT _S A#PUSH(id)#	]j]#	PUSH(id)
#E _S T _S B #AddNext#]EsT _S A	]j]#	LL[A,]=13
#E _S T _S B #AddNext#] EsT _S	]j]#	LL[T _S ,]=6
#E _S T _S B #AddNext#]Es	]j]#	LL[E _S ,]=2
#E _S T _S B #AddNext#]	]j]#	match
#E _S T _S B #AddNext#	]j]#	AddNext

(SUBI, i, 1,t₁)
 (MULTI,t₁,5, t₂)
 (AADD,a,t₂, t₃)
 Pop(2)
 Push(t₃)



- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \varepsilon$
- (3) $Es \rightarrow + T \# \text{GenCode}(+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode}(-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \varepsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode}(*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode}(/) \# Ts$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow id \# \text{Push}(id) \# A$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow \#init\# [E] \#AddNext\# B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E] \#AddNext\# B$

分析栈S

输入流T

动作

#E _S T _S B #AddNext#	[j]#	AddNext
#E _S T _S B	[j]#	LL[B,[]]=16
#E _S T _S B #AddNext#] E[	[j]#	match
#E _S T _S B #AddNext#] E	j]#	LL[E,id]=1
#E _S T _S B #AddNext#] EsT	j]#	LL[T,id]=5
#E _S T _S B #AddNext#] EsT _S P	j]#	LL[P,id]=10
#E _S T _S B #AddNext#] EsT _S V	j]#	LL[V,id]=12

(- , i, 1, t₁)
 (* , t₁, 5, t₂)
 ([] , a, t₂, t₃)
 Pop(2)
 Push(t₃)



- (1) $E \rightarrow T E_s$
- (2) $E_s \rightarrow \varepsilon$
- (3) $E_s \rightarrow + T \# \text{GenCode}(+) \# E_s$
- (4) $E_s \rightarrow - T \# \text{GenCode}(-) \# E_s$
- (5) $T \rightarrow P T_s$
- (6) $T_s \rightarrow \varepsilon$
- (7) $T_s \rightarrow * P \# \text{GenCode}(*) \# T_s$
- (8) $T_s \rightarrow / P \# \text{GenCode}(/) \# T_s$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow id \# \text{Push}(id) \# A$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow \#init\# [E] \#AddNext\# B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E] \#AddNext\# B$

分析栈S

输入流T

动作

#E _S T _S B #AddNext#] EsT _S V	j]#	LL[V,id]=12
#E _S T _S B #AddNext#] EsT _S A#PUSH(id)#id	j]#	match
#E _S T _S B #AddNext#] EsT _S A#PUSH(id)#	]#	PUSH(id)
#E _S T _S B #AddNext#] EsT _S A	]#	LL[A,]=13
#E _S T _S B #AddNext#] EsT _S	]#	LL[T _S ,]=6
#E _S T _S B #AddNext#] Es	]#	LL[E _S ,]=2
#E _S T _S B #AddNext#]	]#	LL[E _S ,]=2

(-, i, 1, t₁)
 (* , t₁, 5, t₂)
 ([] , a, t₂, t₃)
 Pop(2)
 Push(t₃)

j
t ₃

- (1) $E \rightarrow T Es$
- (2) $Es \rightarrow \varepsilon$
- (3) $Es \rightarrow + T \# \text{GenCode}(+) \# Es$
- (4) $Es \rightarrow - T \# \text{GenCode}(-) \# Es$
- (5) $T \rightarrow P Ts$
- (6) $Ts \rightarrow \varepsilon$
- (7) $Ts \rightarrow * P \# \text{GenCode}(*) \# Ts$
- (8) $Ts \rightarrow / P \# \text{GenCode}(/) \# Ts$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow id \# \text{Push}(id) \# A$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow \#init\# [E] \#AddNext\# B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E] \#AddNext\# B$

分析栈S

输入流T

动作

#E _S T _S B #AddNext#]	]#	match
#E _S T _S B #AddNext#	#	AddNext
#E _S T _S B	#	LL[B,#]=15
#E _S T _S	#	LL[T _S ,#]=6
#E _S	#	LL[E _S ,#]=2
#	#	OK

(- , i, 1, t₁)
 (* , t₁, 5, t₂)
 ([], a, t₂, t₃)
 Pop(2)
 Push(t₃)

(- , j, 1, t₄)
 (* , t₄, 1, t₅)
 ([], t₃, t₅, t₆)
 Pop(2)
 Push(t₆)



- (1) $E \rightarrow T E_s$
- (2) $E_s \rightarrow \varepsilon$
- (3) $E_s \rightarrow + T \# \text{GenCode}(+) \# E_s$
- (4) $E_s \rightarrow - T \# \text{GenCode}(-) \# E_s$
- (5) $T \rightarrow P T_s$
- (6) $T_s \rightarrow \varepsilon$
- (7) $T_s \rightarrow * P \# \text{GenCode}(*) \# T_s$
- (8) $T_s \rightarrow / P \# \text{GenCode}(/) \# T_s$
- (9) $P \rightarrow C \# \text{Push}(C) \#$
- (10) $P \rightarrow V$
- (11) $P \rightarrow (E)$
- (12) $V \rightarrow id \# \text{Push}(id) \# A$
- (13) $A \rightarrow \varepsilon$
- (14) $A \rightarrow \#init\# [E] \#AddNext\# B$
- (15) $B \rightarrow \varepsilon$
- (16) $B \rightarrow [E] \#AddNext\# B$

3. 赋值语句的中间代码

- ◆ 赋值语句的形式为: $\text{Left} := \text{Right},$
- ◆ 赋值语句的四元式结构为:
 - Left 的中间代码 //如有必要, 结果在 t_i
 - Right 的中间代码 //如有必要, 结果在 t_j
 - $(\text{FLOAT}, \text{Right}, -, t)$ //如有必要
 - $(:=, \text{Right}/t_j/t, -, \text{Left}/t_i)$

3. 赋值语句的中间代码

- ◆ 赋值语句中间代码生成动作文法如下：

$S \rightarrow V := E \text{ \#Assig\#}$

- ◆ Assig需要做如下处理：

1. 从语义栈中取出赋值号左右分量的语义信息；
2. 比较类型是否相同，如果不同，则生成类型转换中间代码；
3. 生成赋值四元式：
($:=$, Right(t), - , Left)

赋值语句的中间代码的例子

- ◆ `i,j,x,y:integer;`
- ◆ `a : array[1..10][1..5] of integer ;`

赋值语句 `a [ i ] [ j ]:=x+y*3` 的中间代码(其中变量均为整型) :

